AICE2005 - Systematic Design

Formal Specification and Verification

Asieh Salehi Fathabadi
University of Southampton
2025-2026

© University of Southampton

Course Introduction

- Lecture 1: Introduction to Formal Methods
- Lecture 2: Formal Specification Techniques
- Lecture 3: Mathematical Foundations
- Lecture 4: Formal Verification Methods
- Lecture 5: Tools and Applications

© University of Southampton

LECTURE 2:

FORMAL SPECIFICATION TECHNIQUES



612352

© University of Southampton

Recap: Formal Methods (FM)

- **What is a Formal Method (FM):**

  mathematical techniques to specify and verify sw and hw systems
- **Why FM:**

  earlier bug detection
- **When FM:**

  Safety-critical and Security-critical Systems
- **Types of FM:**

  Z, B, Event-B, CPS, TLA, …

© University of Southampton

Lecture 2 Overview

- Review & Specification Fundamentals
- Event-B Modelling (Platform Screen Door Control System)
- B, Z and CSP Review
- Comparison & Selection

Learning Objectives:

- Understand components of formal specifications
- Write basic Event-B specifications
- Model systems using Event-B refinement approach

© University of Southampton

Formal Specification Fundamentals

What is a Formal Specification? "A mathematical description of a system's intended behavior, expressed in precise mathematical notation."

Benefits:

- **Precision**: Eliminate ambiguity in requirements
- Early **Validation**: Stakeholder review before implementation
- **Verification** Basis: Foundation for mathematical proofs
- Test Generation: Automated test case generation
- **Refinement**: Gradual development from abstract to concrete

© University of Southampton

Formal Specification Fundamentals

What is a Formal Specification? "A mathematical description of a system's intended behavior, expressed in precise mathematical notation."

Core Components:

- **State Space**: Define all possible system states
- **Operations**: Define how state changes
- **Invariants**: Properties that always hold
- **Pre/Post-conditions**: Operation constraints

© University of Southampton

Z Notation Overview

What is Z Notation?

- Developed: Oxford University, 1980s
- Foundation: Set theory and first-order predicate logic
- Strength: Rich mathematical notation, data-intensive systems

Basic Z Elements:

- Set membership: $x \in S$
- Power set: $\mathbb{P} S$
- Cartesian product: $A \times B$
- Relation: $A \leftrightarrow B$
- Function: $A \rightarrow B$

```
SchemaName
  declarations    -- variable declarations
  predicates      -- constraints and properties
```


© University of Southampton

Communicating Sequential Processes
CPS Overview

What is CSP?

- Developed: Tony Hoare (1970,-1980)
- Focus: Concurrent systems and communication patterns
- Philosophy: "Processes communicate through synchronized events"
- Applications: Distributed systems, protocols, hardware design

```
Prefix: a → P (Do event a, then behave as P)
Choice: P □ Q (External choice between P and Q)
Sequential: P ; Q (Do P then Q)
Parallel: P || Q (P and Q synchronize on common events)
Interleaving: P ||| Q (P and Q run independently).
```


© University of Southampton

B Method Overview

What is the B Method?

- Developed: Jean-Raymond Abrial (1980s-1990s)
- Foundation: **Set theory, predicate logic, refinement**
- Key Feature: **Stepwise refinement from abstract to concrete**
- Tools: AtelierB, ProB

B Method Philosophy:

- **Abstract** Specification: High-level system behavior
- **Refinement** Steps: Gradual introduction of implementation details
- **Proof Obligations**: Mathematical verification at each step
- Code Generation: Automatic generation from final refinement

© University of Southampton

Event-B Introduction

What is Event-B?

- Evolution: Modern version of B Method (2000s)
- Creator: Jean-Raymond Abrial
- Focus: Event-driven systems, concurrency, distribution
- Tool: **Rodin** Platform (Eclipse-based)
- Strengths: **Refinement, proof automation, model animation**

Event-B Structure:

- **Context**: Static elements (sets, constants, axioms)
- **Machine**: Dynamic elements (variables, invariants, events)
- **Refinement: Step-by-step development**

© University of Southampton

Formal Methods Comparison

Method	Core Idea	System Type	Verification	Strengths	Challenges
Z Notation	Mathematical specification using sets and logic	Data-intensive, sequential	Theorem proving	Precise data modeling	High learning curve
B / Event-B	Set-theoretic models with refinement	Safety-critical, reactive	Proof obligations, model checking	Refinement & tool support (Rodin, ProB)	Requires discipline in proof management
CSP	Processes communicate via events	Concurrent / distributed systems	Model checking (FDR, PAT)	Models concurrency, detects deadlocks	Abstracts away data handling
VDM	State + operation-based modeling	Sequential control systems	Theorem proving / testing	Stepwise refinement to code	Smaller tool ecosystem
TLA+	Temporal logic over states	Distributed protocols, AI coordination	Model checking, theorem proving	Strong for liveness & concurrency	Temporal logic is abstract and symbolic

** This comparison is only a guideline, in practice, the choice of formal method depends on the system's nature, the domain's criticality, and the available expertise.*

© University of Southampton

Case Study: Platform Screen Door Control System

Automated metro platforms use **screen doors** to protect passengers and synchronize with train operations.



** The figure is guided AI-generated*

© University of Southampton

Case Study: Platform Screen Door Control System

System Elements

- **Trains:** approach, stop, and depart from platforms.
- **Platforms:** each has multiple **screen doors**.
- **Controller:** monitors train position and door states.
- **Emergency System:** allows manual intervention in abnormal situations.

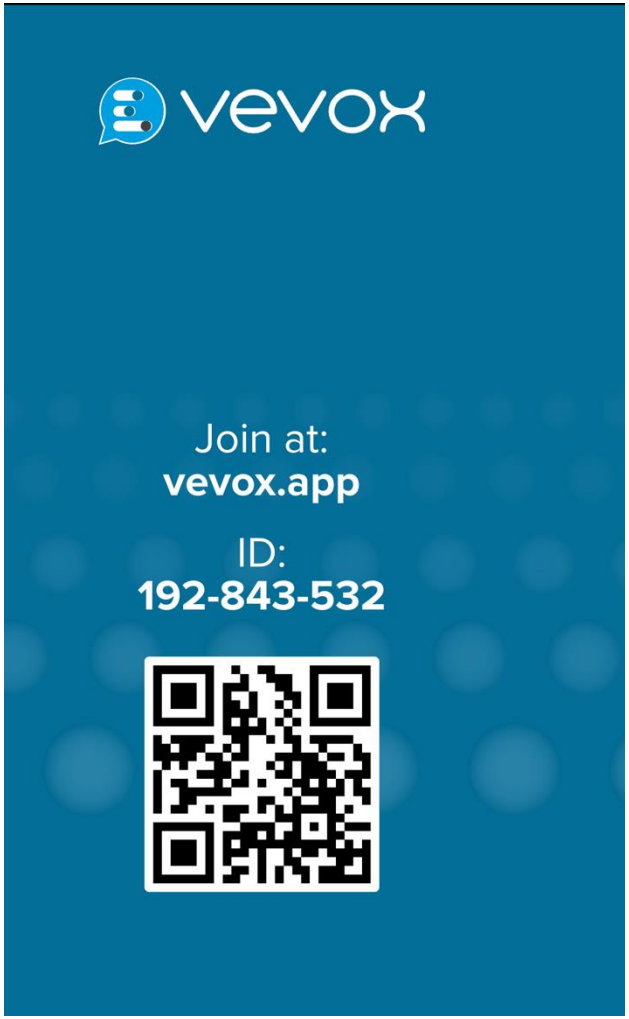
Intended Operation

- Train arrives (detected by sensors).
- Train stops at a designated platform position.
- Platform doors open only when train doors are aligned.
- Before departure, all platform doors must close.
- During emergencies, system must remain in or move to a **safe state**.

© University of Southampton

Let's think and reflect

Safety Requirement Identification



Case Study: Platform Screen Door Control System

System Goal:
Ensure passenger safety at station platforms.

© University of Southampton
University of Southampton

Safety Requirement Identification

- **SR-1**: Doors shall open *only* when a train is stopped at the platform.
- **SR-2**: Doors shall remain *closed* whenever no train is present.
- **SR-3**: Doors shall close before train departure.
- **SR-4**: In emergency mode, doors can be manually overridden.
- **SR-5**: No unsafe or undefined state (e.g., open doors + moving train).

These become your *safety invariants* in Event-B.

© University of Southampton

Event-B Notation Basics

```
CONTEXT  ContextName
SETS
   BasicTypes
CONSTANTS
   constants
AXIOMS
   properties_that_always_hold
END
```

```
MACHINE  MachineName
SEES
   ContextName

VARIABLES
   state_variables
INVARIANTS
   properties_that_always_hold
INITIALISATION
   initial_state_definition
EVENTS
   operation_definitions
END
```


© University of Southampton

Context Definition (Event-B CONTEXT)

```
CONTEXT DoorContext

SETS DOOR_STATE TRAIN_STATE

CONSTANTS OPEN, CLOSED, PRESENT, ABSENT, MOVING
AXIOMS
        partition(DOOR_STATE, {OPEN}, {CLOSED})
        partition(TRAIN_STATE, {PRESENT}, {ABSENT}, {MOVING})

END
```

Defines the static types (sets) and constants

"partition" ensures states are mutually exclusive

© University of Southampton

Abstract Machine: Safety Logic

MACHINE DoorControl

SEES DoorContext

VARIABLES door train

INVARIANTS

door ∈ DOOR_STATE
train ∈ TRAIN_STATE
door = OPEN ⇒ train = PRESENT // SR-1, SR-2
train = MOVING ⇒ door = CLOSED // SR-5

INITIALISATION

door := CLOSED
train := ABSENT

EVENTS

OpenDoors ≙ **when**
train = PRESENT **then**
door := OPEN
end

CloseDoors ≙
any t **where**
door = OPEN
t ∈ TRAIN_STATE **then**
door := CLOSED
end

TrainArrives ≙ **when**
train = ABSENT **then**
train := PRESENT
end

TrainDeparts ≙ **when**
door = CLOSED **then**
train := MOVING
end
END

© University of Southampton

Event-B Structure

- **Context** *(Static, domain facts)*
  - **SETS:** Abstract types
  - **CONSTANTS:** Named domain values
  - **AXIOMS:** Properties over sets/constants (type definitions, constraints)
- **Machine** *(Dynamic, system behavior)*
  - **SEES:** Links to one or more Contexts
  - **VARIABLES (state):** System state components
  - **INVARIANTS:** Properties that **must always hold** for the variables, before and after every event
- **INITIALISATION:** Assigns the initial state satisfying the invariants
- **EVENTS:** Possible state transitions *(How behavior is modelled)*
  - **Guards:** Conditions enabling the event (when it is allowed)
  - **Actions:** State updates
  - **ANY / WHERE:** (optional) Binds local parameters with constraints

© University of Southampton

Abstract Machine: Safety Logic

MACHINE DoorControl

Events model possible behaviors of the system

SEES DoorContext

door ∈ DOOR_STATE

train ∈ TRAIN_STATE

door = OPEN ⇒ train = PRESENT // SR-1, SR-2

train = MOVING ⇒ door = CLOSED // SR-5

`OpenDoors` can only fire when `train = PRESENT`

The 'then' clauses change the state of door to 'open'

SR-1 & SR-2: Ensures doors stay closed when no train

SR-5: Prevents the unsafe state of open doors + moving train

train := ABSENT

EVENTS

OpenDoors ≙ **when**

train = PRESENT then

door := OPEN

any t **where**

t ∈ TRAIN_STATE **then**

door := CLOSED

end

then

train := PRESENT

end

TrainDeparts ≙ **when**

door = CLOSED **then**

train := MOVING

end

END

© University of Southampton

University of Southampton

Refinement (Adding Implementation Detail)

```
MACHINE DoorControl_R1 REFINES DoorControl
VARIABLES door train sensor
INVARIANTS
        sensor ∈ BOOL
        sensor = TRUE ⇔ train ∈ {PRESENT}
EVENTS
OpenDoors refines OpenDoors ≙ when
        sensor = TRUE
        train = PRESENT then
        door := OPEN end
CloseDoors refines CloseDoors ≙ any t where
        door = OPEN
        t ∈ TRAIN_STATE then
        door := CLOSED end
TrainArrives refines TrainArrives ≙ when
        train = ABSENT then
        train := PRESENT end
TrainDeparts refines TrainDeparts ≙ when
        door = CLOSED then
        train := MOVING
        sensor := FALSE end
DetectTrain ≙ when  // new event
        sensor = FALSE
        train = PRESENT then
        sensor := TRUE end
END
```

- Introduces **sensor variable** representing physical train detection
- **Gluing invariant** links abstract and concrete variables
- Events refined to include sensor checks
- New event "DetectTrain" models sensor activation
- Rodin tool will generate proof obligations to verify safety is preserved (more on Lecture 4)

© University of Southampton

Summery

- **Overview of :** Z, CPS, B, Event-B
- **Safety Requirement** identification
- **Safety properties** specification
- **First Event-B model** construction

Let's read and explore

- Modeling in Event-B: System and Software Engineering
  by Jean-Raymond Abrial
- https://wiki.event-b.org/
- Systems Engineering Principles and Practice, Alexander Kossiakoff et. al., 2011

© University of Southampton